Data transformation

Marcin Włodarczak

15 November 2024

We start by loading `tidyverse` and `languageR`.

```
library('tidyverse')
theme_set(theme_bw())
library('languageR')
```

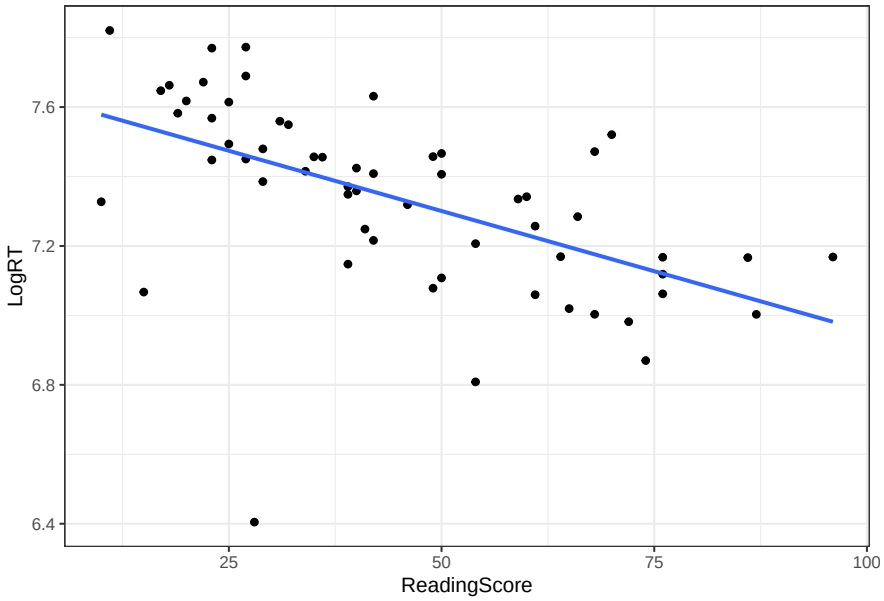
Centering

Here is the `beginningReaders` model from last time:

```
data('beginningReaders')

readers <- beginningReaders %>%
  group_by(Subject) %>%
  summarise(LogRT = mean(LogRT),
            ReadingScore = first(ReadingScore))

m <- lm(LogRT ~ ReadingScore, readers)
summary(m)
```

```
##
## Call:
## lm(formula = LogRT ~ ReadingScore, data = readers)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.04864 -0.06365  0.03319  0.12060  0.35863
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   7.647536   0.068506 111.633  < 2e-16 ***
## ReadingScore -0.006936   0.001387  -5.001 5.77e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2241 on 57 degrees of freedom
## Multiple R-squared:  0.305,  Adjusted R-squared:  0.2928
## F-statistic: 25.01 on 1 and 57 DF,  p-value: 5.769e-06
```

```
ggplot(readers, aes(ReadingScore, LogRT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

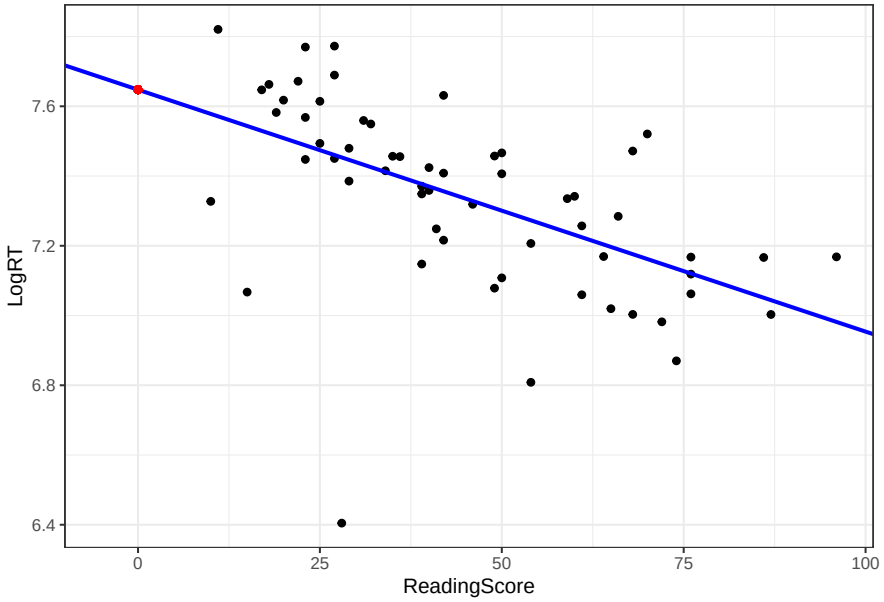


Let's extract the model coefficients:

```
b0 <- coefficients(m)[1]
b1 <- coefficients(m)[2]
```

What does the intercept correspond to in this model? (Do not worry about the details of the code below, this is mainly for illustrative purposes.)

```
ggplot(readers, aes(ReadingScore, LogRT)) +
  geom_point() +
  geom_abline(slope = b1, intercept = b0, color = 'blue', linewidth = 1) +
  geom_point(x = 0, y = b0, color = 'red') +
  xlim(-5, max(readers$ReadingScore))
```



Clearly, the intercept is completely meaningless in this case - it corresponds to the expected reaction time for a child with a reading score value of 0, which makes no sense at all.

Wouldn't it be nice to put the intercept to some good use, though? There is a simple trick – since the intercept corresponds to the value of the dependent variable when the independent variable equals 0, we can change the meaning of the intercept by transforming the independent variable such than 0 corresponds to some value of interest.

For instance, if we want the intercept to correspond to the expected reaction time for the average reading score in the sample, we can simply subtract the mean reading score from each data point. This is known as *centering*:

```
readers <- readers %>%
  mutate(ReadingScore_c = ReadingScore - mean(ReadingScore))
```

The mean of the transformed variable is now effectively 0:

```
round(mean(readers$ReadingScore_c), 10)
```

```
## [1] 0
```

Note that all this does is shift the f_0 distribution, it does not affect its spread:

```
sd(readers$ReadingScore)
```

```
## [1] 21.2126
```

```
sd(readers$ReadingScore_c)
```

```
## [1] 21.2126
```

Let's refit the model using the centered independent variable:

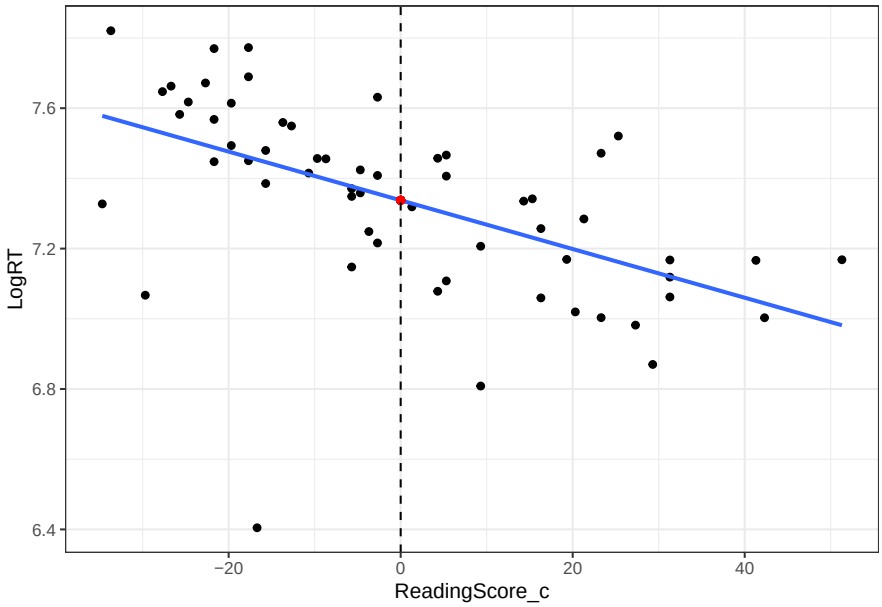
```
m_c <- lm(LogRT ~ ReadingScore_c, readers)
coefficients(m_c)
```

```
##    (Intercept) ReadingScore_c
##    7.337517241   -0.006936342
```

The intercept now corresponds to the expected reaction time for mean reading score:

```
ggplot(readers, aes(ReadingScore_c, LogRT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE) +
  geom_point(x = 0, y = coef(m_c)[1], color = 'red') +
  geom_vline(xintercept = 0, linetype = 2)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



We can make sure that this is the case by getting the prediction for the mean reading score in the original model:

```
b0 + b1 * mean(readers$ReadingScore)
```

```
## (Intercept)
##    7.337517
```

Finally, note that because we have not altered the spread of the independent variable, the estimate of slope has remained unchanged:

```
coefficients(m)
```

```
##  (Intercept) ReadingScore
##  7.647536473 -0.006936342
```

```
coefficients(m_c)
```

```
##    (Intercept) ReadingScore_c
##    7.337517241   -0.006936342
```

Exercise

Here is the model predicting SPL from f_0 you worked with last time:

```
voice <- read_csv('f0-spl.csv')
m_voice <- lm(spl ~ f0, voice)
```

1. Interpret coefficients in the original model.
2. Center the independent variable and refit the model.
3. Re-interpret the coefficients in the refitted model.

```
# Write your solution here.
```

Standardising

Since reading score is expressed on a scale from 0 to 100, the value of slope is the change in reaction time (expressed in $\log(ms)$) corresponding to one percentage point increase of reading score:

```
b1
```

```
## ReadingScore
## -0.006936342
```

Let's express reading score on a scale from 0 to 1 instead:

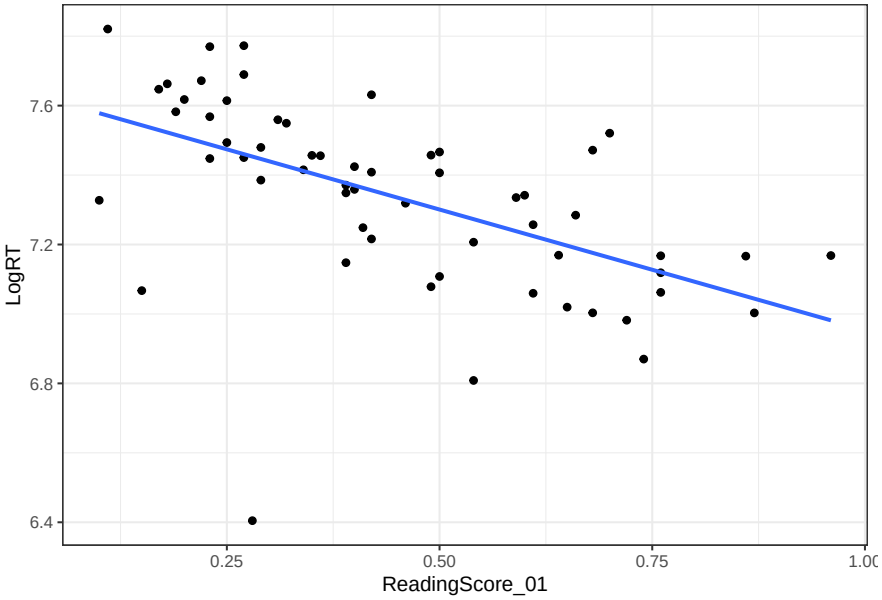
```
readers <- readers %>%
  mutate(ReadingScore_01 = ReadingScore / 100)
range(readers$ReadingScore_01)
```

```
## [1] 0.10 0.96
```

Let's refit the model using this predictor:

```
ggplot(readers, aes(ReadingScore_01, LogRT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
m_01 <- lm(LogRT ~ ReadingScore_01, readers)
```

After this transformation, a unit increase of reading score now spans the whole scale! Since we have compressed the scale by a factor of 100, the value of slope has been inflated by the same factor:

```
coefficients(m_01)
```

```
##     (Intercept) ReadingScore_01
##       7.6475365      -0.6936342
```

Of course, this does not mean that the effect itself is 100 times larger, though! For this reason, we might want to express our independent variables in units of standard deviation. We can achieve this by *dividing* reading score values by their standard deviation:

```
readers <- readers %>%
  mutate(ReadingScore_1sd = ReadingScore / sd(ReadingScore))
```

We can check the the standard deviation of the transformed variable is indeed equal to 1:

```
sd(readers$ReadingScore_1sd)
```

```
## [1] 1
```

Most commonly, this transformation is combined with centering and is known as *standardisation*, *z-normalisation* or *z-scoring*:

```
readers <- readers %>%
  mutate(ReadingScore_z = (ReadingScore - mean(ReadingScore)) / sd(ReadingScore))
```

Our independent variable now has a mean of 0 and standard deviation of 1:

```
round(mean(readers$ReadingScore_z), 10)
```

```
## [1] 0
```

```
sd(readers$ReadingScore_z)
```

```
## [1] 1
```

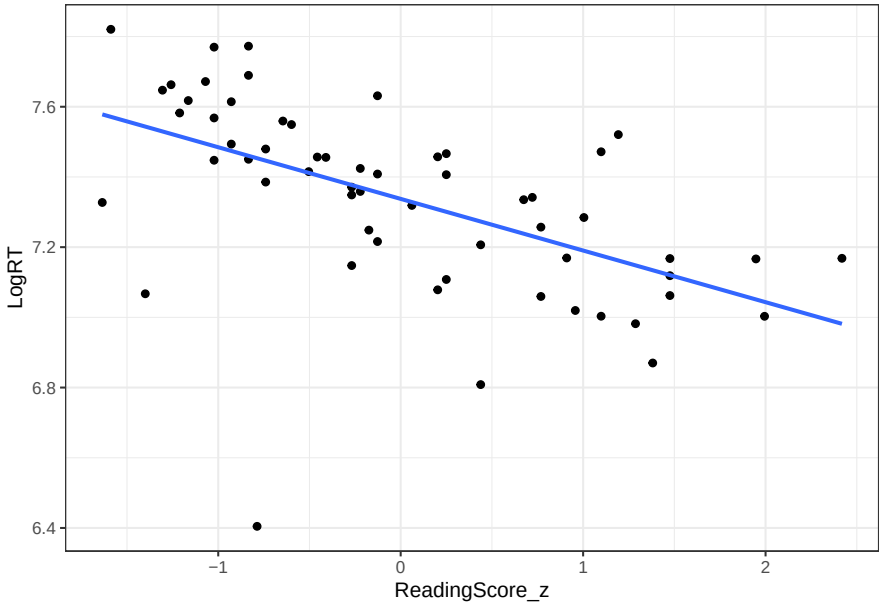
What is the interpretation of slope and intercept in this model?

```
m_z <- lm(LogRT ~ ReadingScore_z, readers)
coef(m_z)
```

```
##    (Intercept) ReadingScore_z
##      7.3375172     -0.1471378
```

```
ggplot(readers, aes(ReadingScore_z, LogRT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Exercise

Refit the SPL/f_0 model after z-scoring the independent variable. Interpret the coefficients.

```
# Write your solution here.
```

Correlation

Note that when both the independent and the dependent variables are standardised, the value of slope is equal to Pearson's correlation coefficient r:

```
readers <- readers %>%
  mutate(LogRT_z = (LogRT - mean(LogRT)) / sd(LogRT))

m_zz <- lm(LogRT_z ~ ReadingScore_z, readers)
coefficients(m_zz)[2]
```

```
## ReadingScore_z
##     -0.5522666
```

```
cor(readers$LogRT, readers$ReadingScore)
```

```
## [1] -0.5522666
```

We can now re-appreciate our understanding of the correlation coefficient – it corresponds to the effect of x on y when both variables are standardised (i.e. when their standard deviations are equal to 1). This also makes r a standardised measure of the strength of a relationship in the sense that its magnitude does not depend on the metric used to measure x and y.

Since we are dealing with a simple regression problem, we can also note that the coefficient of determination (r^2) is equal to the square of r:

```
cor(readers$LogRT, readers$ReadingScore)^2
```

```
## [1] 0.3049984
```

```
summary(m_zz)$r.squared
```

```
## [1] 0.3049984
```

Remember, however, that this is not going to be the case once you have more than one predictor!

Log-transformation

Let's get a reaction times and word frequencies for one speaker (`s15`) in the `beginningReaders` data set:

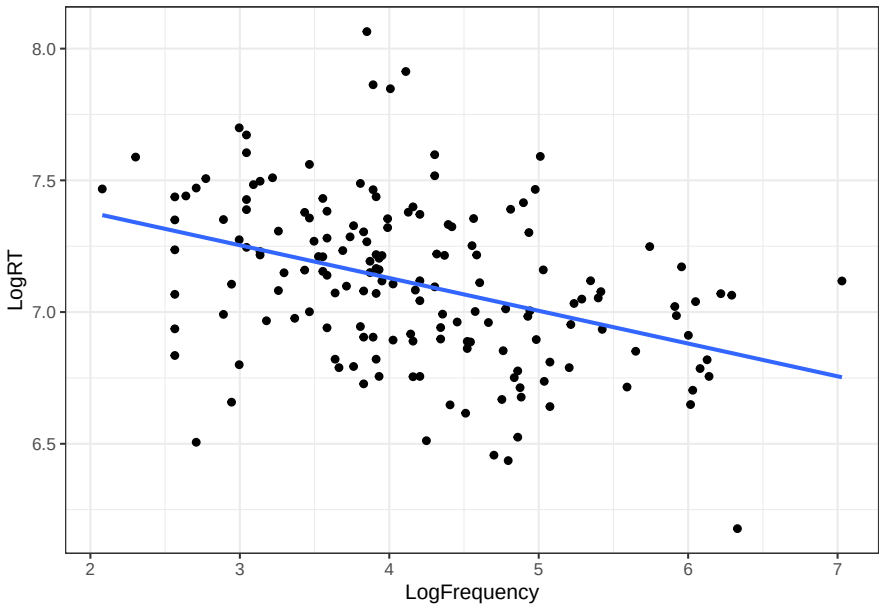
```
s15 <- beginningReaders %>%
  filter(Subject == 'S15') %>%
  select(LogRT, LogFrequency, Word)

head(s15)
```

```
##        LogRT LogFrequency     Word
## 16  7.332369     4.394449 avontuur
## 67  7.440734     2.639057    baden
## 120 7.285507     3.737670   balkon
## 159 7.160069     5.030438     band
## 200 7.105786     2.944439  barsten
## 258 7.252054     4.553877     beek
```

We can fit a regression model predicting reaction time for words with different frequencies. As expected, reaction times get shorter as words get more frequent:

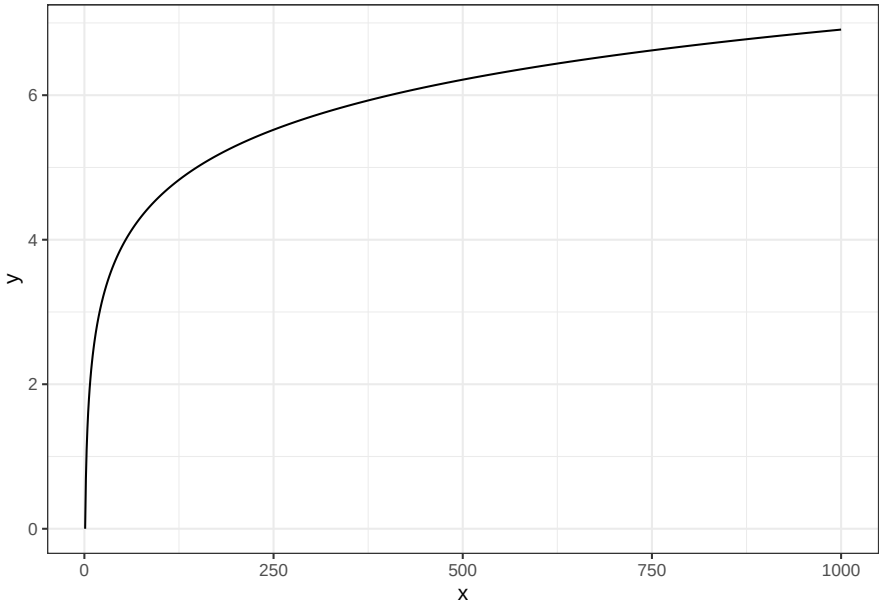
```
ggplot(s15, aes(LogFrequency, LogRT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Note that both variables are expressed on a logarithmic scale (in this case using natural logarithm). Here is the natural logarithm function in all its glory:

```
log_fn <- tibble(x = 1:1000,
                 y = log(x))

ggplot(log_fn, aes(x, y)) +
  geom_line()
```



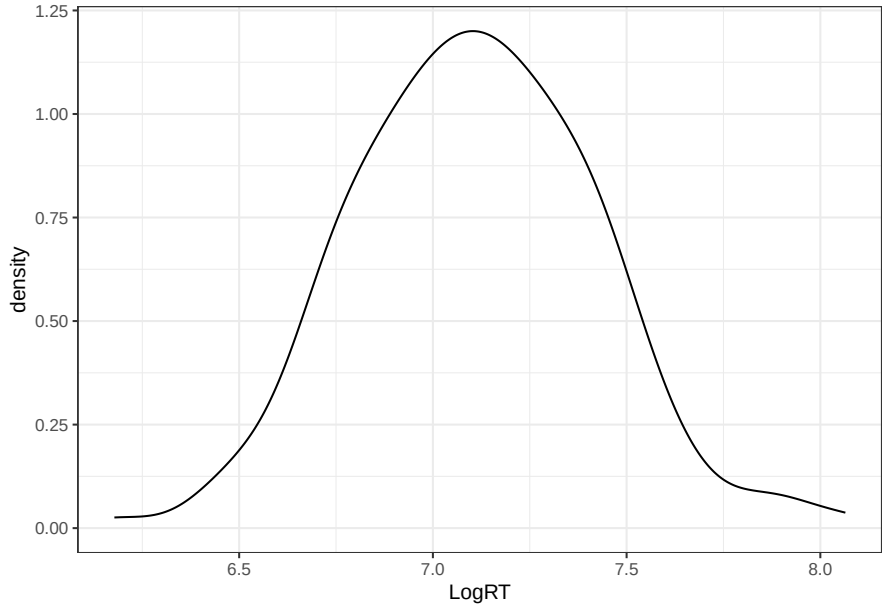
Exercise

Produce analogues plots for the $\log_2$ and $\log_{10}$ functions and compare their shapes. Use the `log2` and `log10` functions in R.

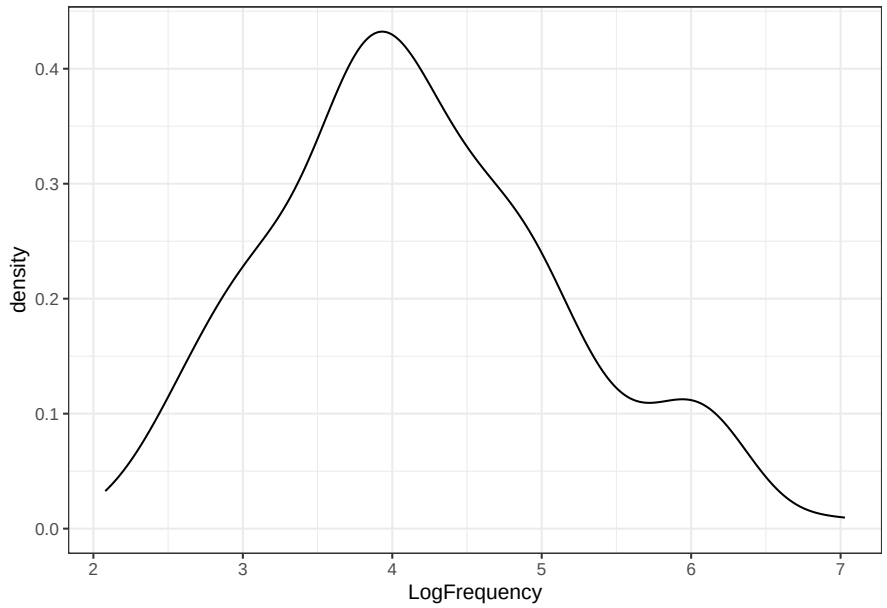
```
# Write your solution here.
```

All logarithmic scales have the effect of compressing large numbers more than small numbers, which produces reasonably symmetric distributions of `LogRT` and `LogFrequency`:

```
ggplot(s15, aes(LogRT)) +
  geom_density()
```




```
ggplot(s15, aes(LogFrequency)) +
  geom_density()
```



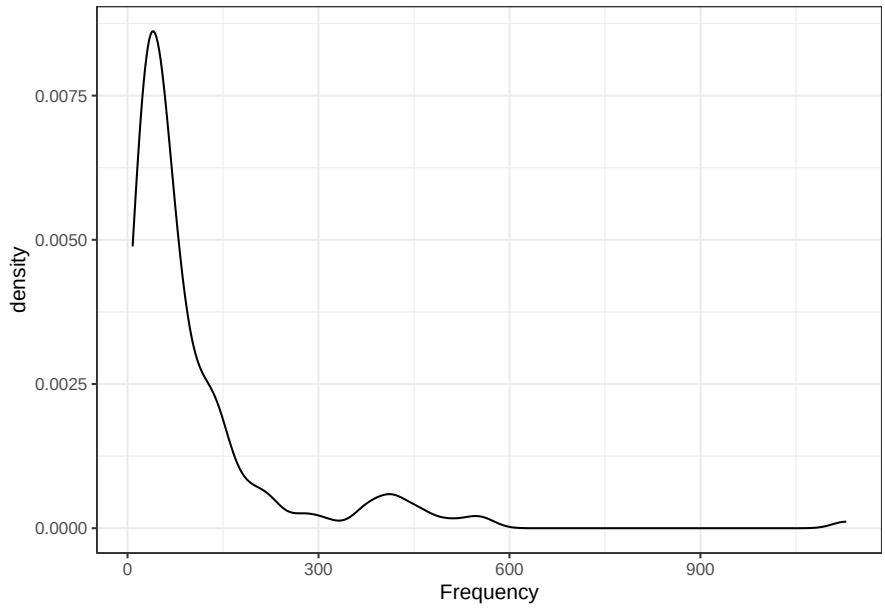
Let's now have a look at the non-transformed values of these variables. Since they were transformed using

the `log` function (i.e. the natural logarithm), we can convert them back to milliseconds and raw counts using the `exp` function:

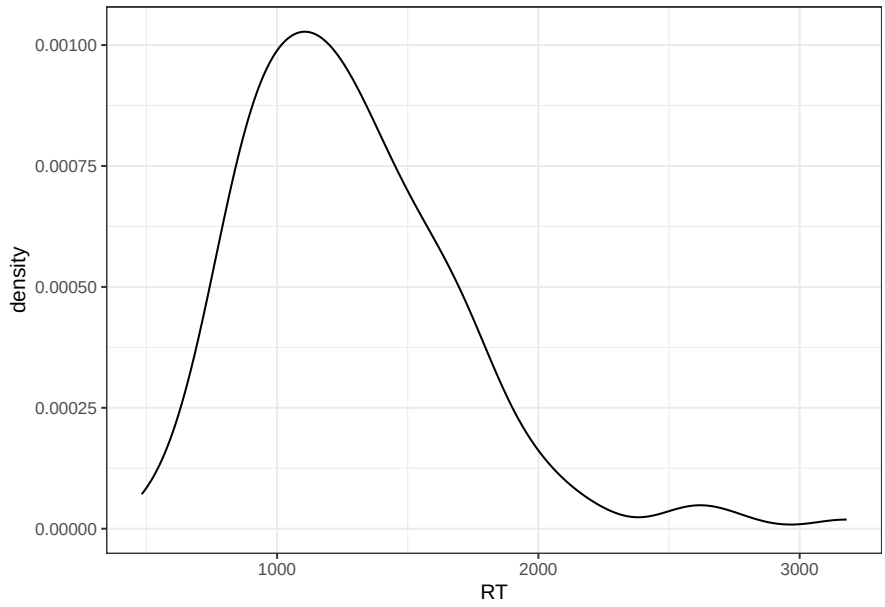
```
s15 <- s15 %>%
  mutate(RT = exp(LogRT),
         Frequency = exp(LogFrequency))
```

When plotted on linear scale, the variables show positive skew, which the log-transformation successfully dealt with:

```
ggplot(s15, aes(Frequency)) +
  geom_density()
```



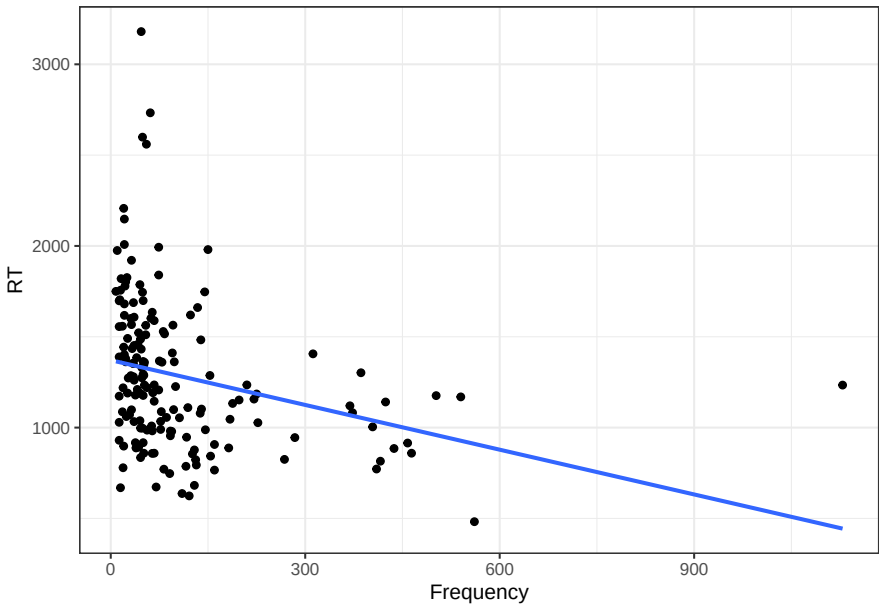

```
ggplot(s15, aes(RT)) +
  geom_density()
```



Using the non-transformed variables in a regression model produces the following result:

```
ggplot(s15, aes(Frequency, RT)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Clearly, log-transforming the variables improves model fit but how do we interpret the parameters when variables are expressed on the logarithmic scale?

```
m_s15 <- lm(LogRT ~ LogFrequency, s15)
b0_s15 <- coefficients(m_s15)[1]
b1_s15 <- coefficients(m_s15)[2]
```

We can of course interpret the model using the log-transformed variables, that is in terms of log-seconds and log-counts. However, that is not very informative. After all, what is a decrease of reaction time equal to -0.12 $\log(ms)$. Unfortunately, translating coefficients from logarithmic to linear scale is a bit tricky since it involves switching between addition and multiplication.

Winter proposes a good alternative strategy:

1. Pick two values of the independent variable on linear scale.
2. If the independent variable is modeled on log-scale transform the values using the correct logarithmic function, e.g. $\log_2$, $\log_{10}$, or log.
3. Calculate the expected values.
4. If the dependent variable is log-transformed, convert the expected values back to linear scale. For natural logarithm, use `exp()`, for $\log_2$ and $\log_{10}$, use powers of 2 and 10, respectively.

For instance, in our model we could get predictions for the words "raket" (*rocket*) and "broer" (*brother*). The frequncies for these words are as follows:

```
s15 %>%
  filter(Word %in% c('raket', 'broer')) %>%
  select(Word, Frequency)
```

```
##       Word Frequency
## 721  broer       386
## 5011 raket        36
```

We can convert them to log scale:

```
broer_LogFreq <- log(386)
raket_LogFreq <- log(36)
```

Now we can calculate the expected values:

```
broer_LogRT <- b0_s15 + b1_s15 * broer_LogFreq
raket_LogRT <- b0_s15 + b1_s15 * raket_LogFreq
```

Finally, we can convert the expected values back to linear scale:

```
exp(broer_LogRT)
```

```
## (Intercept)
##    978.4495
```

```
exp(raket_LogRT)
```

```
## (Intercept)
##     1314.33
```

Exercise

Let's transform the data using the $\log_2$ function instead:

```
s15 <- s15 %>%
  mutate(Log2Frequency = log2(Frequency),
         Log2RT = log2(RT))
```

Refit the model using these variables:

```
# Write your solution here.
```

Next, interpret the model by calculating reaction times (in ms) for the words "garage" and "minuut":

Hint: Here is how to convert a $\log_2$ value back to linear scale:

```
x <- c(5, 7.6, 8.9)
x_log2 <- log2(x)
2^x_log2
```

```
## [1] 5.0 7.6 8.9
```

```
# Write your solution here.
```